Models and Tools for Cyber-Physical Systems

Digital Written Exam, June the 12th 2023, 10:00-14:00

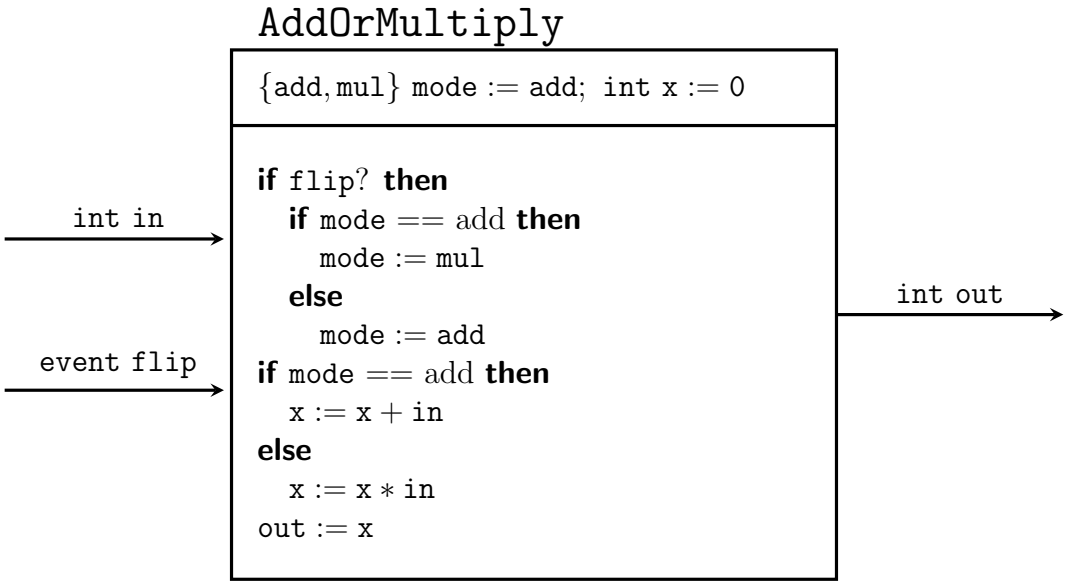
Please read the following before solving the exercises.

- This exam contains 6 exercises, worth 75 points in total. Each exercise is compulsory. The solution has to be composed in English. If you believe that the assignment wording is ambiguous or erroneous, then write down what additional assumption you are using and outline your reasons.
- In some exercises you are asked to extend incomplete UPPAAL and Python files. Solutions have to be uploaded to Digitaleksamen in the form of UPPAAL, Python, pdf or text files. For each of your solutions, ensure that it is clear which exercise is solved.
- The exam is open book but asking/receiving help from others is not allowed.
- In case of emergencies: Students can contact the instructors during the exam by approaching the study secretary.

Last but not least, good luck!

Exercise 1: Synchronous Components 10 Points

Consider the synchronous component `AddOrMultiply` given as a block diagram below.



(a) Below we wrote the initial state and a sequence of inputs. Fill in the blanks to obtain a proper prefix of a corresponding execution.

$$(\texttt{add}, 0) \xrightarrow{(2,\bot)/} \underline{\qquad\qquad} \xrightarrow{(3,\top)/} \underline{\qquad\qquad}$$

$$\xrightarrow{(1,\top)/} \underline{\qquad\qquad} \xrightarrow{(3,\bot)/} \underline{\qquad\qquad}$$

Hint: Recall that $\bot$ denotes that an event is not present, and $\top$ denotes that it is present. You need to provide both the missing states between the arrows and the missing outputs after the "/" on the arrows.

(b) Give a <u>sequence of inputs</u> leading to the state $(\texttt{mul}, 42)$.
(You <u>do not</u> have to provide the intermediate states and outputs.)

(c) Describe the component `AddOrMultiply` as an extended state machine.
Use the mode of the state machine to capture the value of the state variable `mode`, while the state variable `x` should be updated using assignments in the mode switches.

Hint: Recall that `flip?` is used to check that the `flip` event is present, and $\neg$`flip?` to check that it is absent.

. . . . . Solution sketch . . . . . .

(a) $(\texttt{add}, 0) \xrightarrow{(2,\bot)/2} (\texttt{add}, 2) \xrightarrow{(3,\top)/6} (\texttt{mul}, 6) \xrightarrow{(1,\top)/7} (\texttt{add}, 7) \xrightarrow{(3,\bot)/10} (\texttt{add}, 10)$

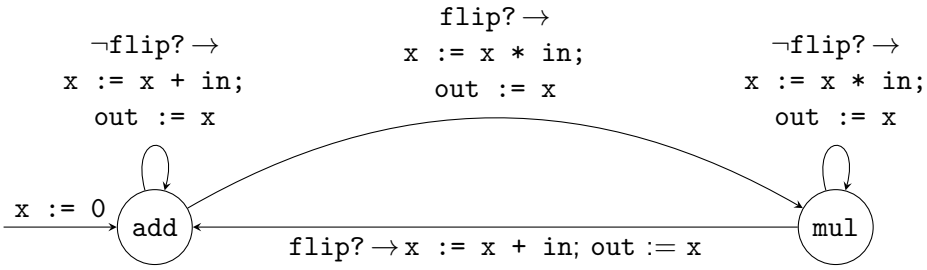
(b) The shortest sequences have length two (because we cannot start with `mul` but need it eventually). Below are all valid length-two solutions.

$$(42, \bot), (1, \top) \qquad (21, \bot), (2, \top) \qquad (14, \bot), (3, \top) \qquad (7, \bot), (6, \top)$$
$$(6, \bot), (7, \top) \qquad (3, \bot), (14, \top) \qquad (2, \bot), (21, \top) \qquad (1, \bot), (42, \top)$$

Execution corresponding to the first solution:

$$(\texttt{add}, 0) \xrightarrow{(42,\bot)/42} (\texttt{add}, 42) \xrightarrow{(1,\top)/42} (\texttt{mul}, 42)$$

(c)



Exercise 2: Inductive Invariants 10 Points

Consider a transition system T with two integer variables `x` and `y` and a Boolean variable `z`. All variables are initially 1. The transitions of T correspond to executing the following conditional statement:

```
if (z = 1) then {x := x * 2; z := 0} else {y := y * 2; z := 1}
```

Consider the following properties:

$\varphi_1 : x$ is even and y is even (formally: $\exists k_x, k_y \in \mathbb{Z}.\ x = 2k_x \wedge y = 2k_y$)
$\varphi_2 : x = y \vee x = 2y$

(a) Argue why φ_1 is not an inductive invariant of the transition system T.

(b) Argue why φ_2 is not an inductive invariant of the transition system T.

(c) Provide an inductive invariant of the transition system T.
(You <u>do not</u> have to argue why your solution is an inductive invariant.)

.....Solution sketch .....

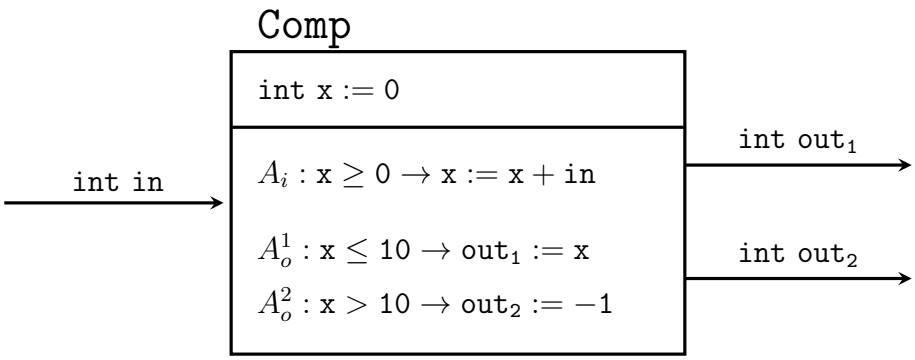
(a) φ_1 is not an invariant because it does not contain the initial state $x = y = z = 1$.

(b) φ_2 is not inductive because the state $x = 1, y = 1, z = 0$ is contained, but the successor state $x = 1, y = 2, z = 1$ is not.

(c) Possible solution: $(x = y \wedge z = 1) \vee (x = 2y \wedge z = 0)$

Exercise 3: Asynchronous Components 10 Points

Consider the following asynchronous component `Comp` given as a block diagram.



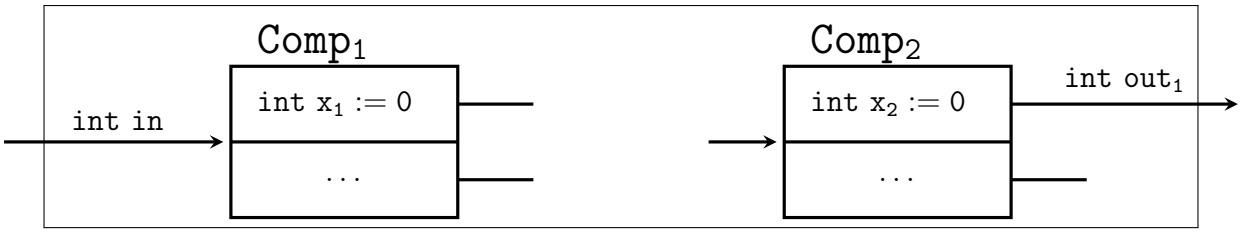
Consider the parallel composition given by the following description:

$$\underbrace{\texttt{Comp}[\texttt{out}_1 \mapsto \texttt{out}_0][\texttt{out}_2 \mapsto \texttt{temp}]}_{\texttt{Comp}_1} \;\mid\; \underbrace{\texttt{Comp}[\texttt{in} \mapsto \texttt{temp}]}_{\texttt{Comp}_2} \;\setminus\; \texttt{temp}$$

(a) Below is a partial block diagram for the parallel composition. Connect the inputs and outputs of the components correctly with edges/arrows. Inputs and outputs of the composition should leave the outer block; we have given an example for each.

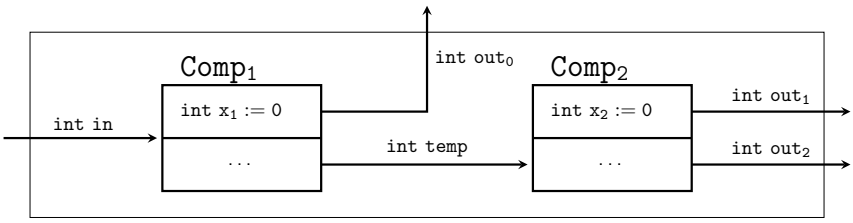
(b) Annotate the edges you drew with the channel types and names.

(c) List all input tasks, internal tasks, and output tasks of the parallel composition.



Solution sketch

(a) and (b)



(c) The internal task is defined syntactically, but could be simplified to a single statement.

input: $A_i : \texttt{x}_1 \geq 0 \rightarrow \texttt{x}_1 := \texttt{x}_1 + \texttt{in}$

internal: $A_t : \texttt{x}_1 > 10 \wedge \texttt{x}_2 \geq 0 \rightarrow \underbrace{\{\texttt{temp} := -1; \texttt{x}_2 := \texttt{x}_2 + \texttt{temp}\}}_{\text{(simplified:) } \texttt{x}_2 := \texttt{x}_2 - 1}$

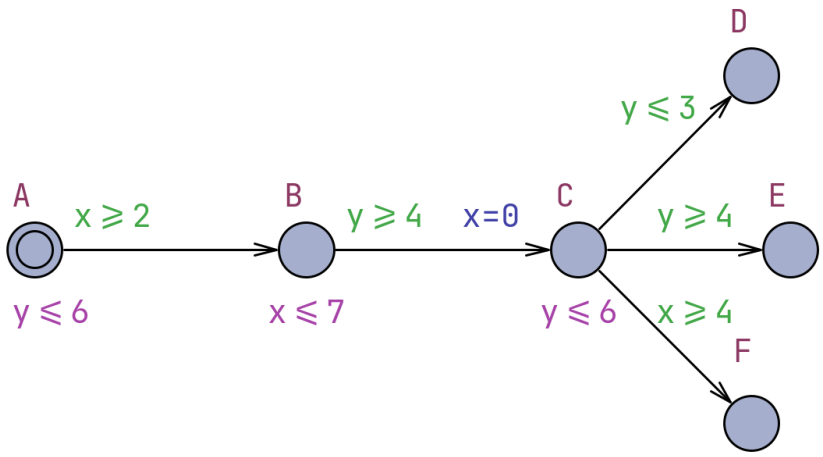
output: $A_o^1 : \texttt{x}_1 \leq 10 \rightarrow \texttt{out}_0 := \texttt{x}_1$

$A_o^2 : \texttt{x}_2 \leq 10 \rightarrow \texttt{out}_1 := \texttt{x}_2$

$A_o^3 : \texttt{x}_2 > 10 \rightarrow \texttt{out}_2 := -1$

Exercise 4: Exploration of Timed Automata 10 Points

Consider the timed automaton $\mathcal{A}$ below with two clocks x and y.



(a) Which of the three locations D, E and F are reachable from the initial state (A, $\mathtt{x} = 0, \mathtt{y} = 0$)?

(b) For each of these three goal locations that is reachable, provide a timed transition sequence that leads to the location from the initial state.

(c) For each of the three goal locations that is reachable, what is the fastest time of reaching that location. Provide a witness timed transition sequence.

(d) Describe using difference constraints the reachable zones upon entry and after delay for the locations A, B and C.

(e) For each of the three goal locations that is NOT reachable, investigate whether it is possible to weaken the guard leading to the location, so that the location becomes reachable.
NOTE: `x<=7` is weaker than `x<=5`. Similarly, `x>=2` is weaker than `x>=4`.

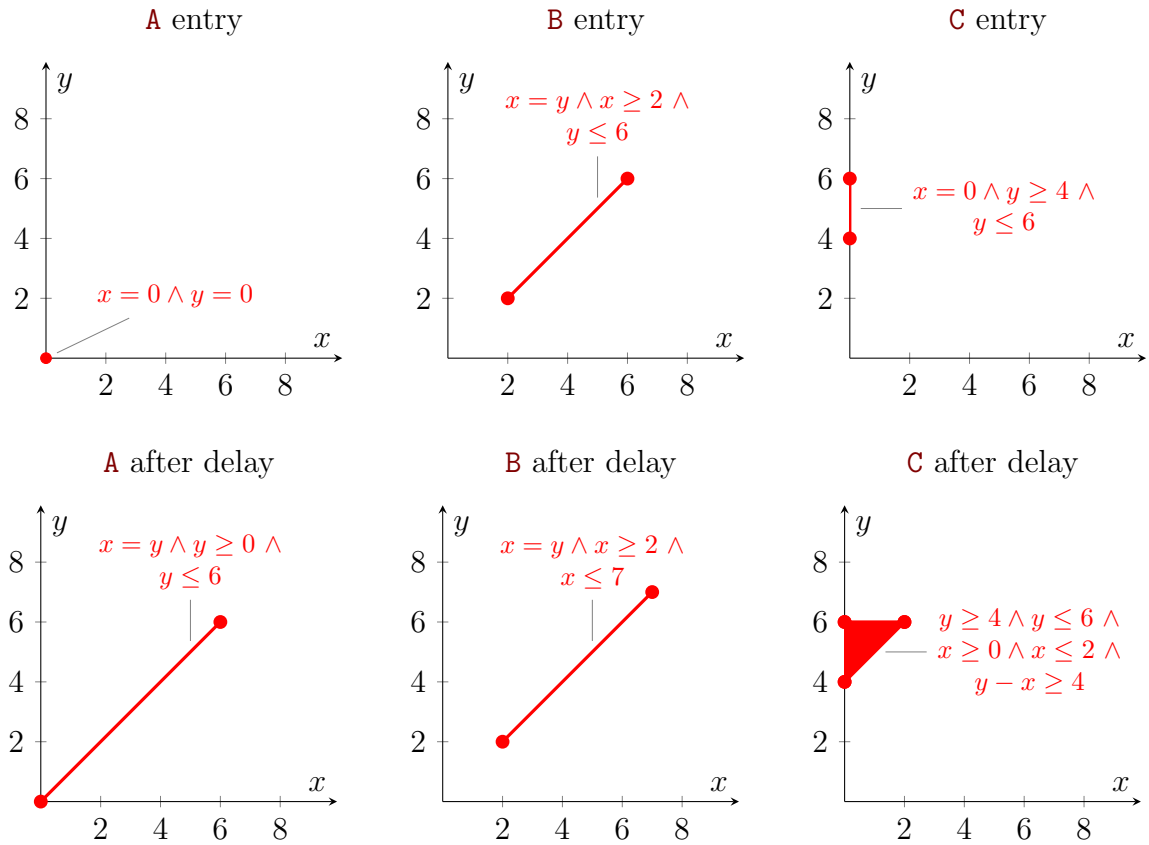
Solution sketch

(a) Only E.

(b) $(\mathtt{A}, x = 0, y = 0) \xrightarrow{2} (\mathtt{A}, x = 2, y = 2) \rightarrow (\mathtt{B}, x = 2, y = 2) \xrightarrow{2} (\mathtt{B}, x = 4, y = 4) \rightarrow (\mathtt{C}, x = 0, y = 4) \rightarrow (\mathtt{E}, x = 0, y = 4)$

(c) The fastest time is 4 time units. The witnessing sequence is above.

(d) The reachable zones upon entry and after delay:



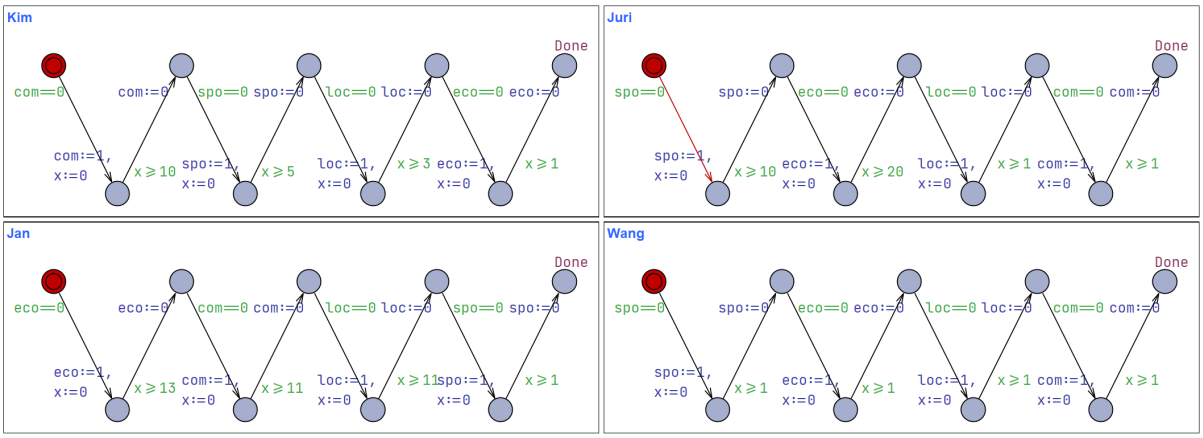
(e) Changing the guard to $y \leq 4$ will make D reachable. Changing the guard to $x \geq 2$ will make F reachable.

Exercise 5: Sharing a Newspaper 15 Points

Here we consider the problem of how to optimize the reading of a newspaper with different sections but shared between several people. In our concrete example, 4 people (Kim, Juri, Jan and Wang) are sharing the newspaper having 4 sections (sport, economy, local news, comic strip). Each person has a preferred ordering of reading the sections and a required minimum reading time for each section. The table below gives the reading-order and -time information with each cell giving the reading number and reading time for the given person/section (e.g. Kim wants to read the comic strip as the first section for at least 10 minutes).

	Sport	Economy	Local News	Comic Stip
Kim	2. 5 min	4. 1 min	3. 3 min	1. 10 min
Jüri	1. 10 min	2. 20 min	3. 1 min	4. 1 min
Jan	4. 1 min	1. 13 min	3. 11 min	2. 11 min
Wang	1. 1 min	2. 1 min	3. 1 min	4. 1 min

Now the key question is how to optimally *schedule* the reading of the sections so that the total time until all persons have completed reading all sections is minimal? Here a schedule S gives for each person p and section s the times $S(p, s)$ and $E(p, s)$, when person p starts and ends reading section s. Of course the preferences (reading-orders and -times) must be met. Also, each section can be read by at most one person at the time, however different sections can be read by different persons in parallel.
Below you can find the screenshot of a UPPAAL model of the problem. Here each person is modelled as a timed automaton with a sequence of transitions reflecting the reading order. The minimum reading times are modelled using constraints on a local clock. The fact that sections can at most be read by one person is modelled using 4 Boolean variables used as semaphores. You can find the complete UPPAAL model in Digital Exam in the file `Newspaper.xml`.



Now answer the following questions:

(a) Formulate as a UPPAAL query ϕ_1 that all 4 persons can complete the reading (according to the ordering and timing constraints) within 50 minutes.

(b) Check in UPPAAL whether ϕ_1 is satisfied, and if so, provide a schedule meeting the overall deadline 50. You can illustrate the schedule using an Message Sequence Chart (MSC) chart for ϕ_1.

Hint: In the symbolic simulator of UPPAAL, you can save the generated MSC by right-clicking it and select "Export".

(c) Formulate as a UPPAAL query ϕ_2 that all 4 persons can complete the reading (according to the ordering and timing constraints) within 20 minutes.

(d) Check in UPPAAL whether ϕ_2 is satisfied, and if so, provide a schedule meeting the overall deadline 20. You can illustrate the schedule using an MSC chart for ϕ_2.

(e) Use UPPAAL to find the minimum completion time. State what it is and give an optimal schedule achieving this minimum. You can illustrate the optimal schedule using an MSC chart obtained from UPPAAL.

In the following modify the model in `Newspaper.xml` in four different (independent) ways.

(f) In the original model above Wang wants to spend a minimum of 1 minute on reading each section. Change these reading times to sport:2min, economy:3min, local news: 4min, and comic strip:5min. Modify the original model `Newspaper.xml` accordingly, and use UPPAAL to find the minimum completion time under this modification.

Please upload to Digital Exam your modified UPPAAL model in a file with name `Newspaper2.xml`.

(g) Modify the original model `Newspaper.xml` so that the reading order of Kim is reversed, i.e. economy is first section to read and comic strip is the last section. Use UPPAAL to find the minimum completion time under this modification.

Please upload to Digital Exam your modified UPPAAL model in a file with name `Newspaper3.xml`.

(h) Modify the original model `Newspaper.xml` so that Jan is not allowed to start reading before Kim has read the first two sections (i.e. has completed reading the sport section). Use UPPAAL to find the minimum completion time under this modification.

Please upload to Digital Exam your modified UPPAAL model in a file with name `Newspaper4.xml`.

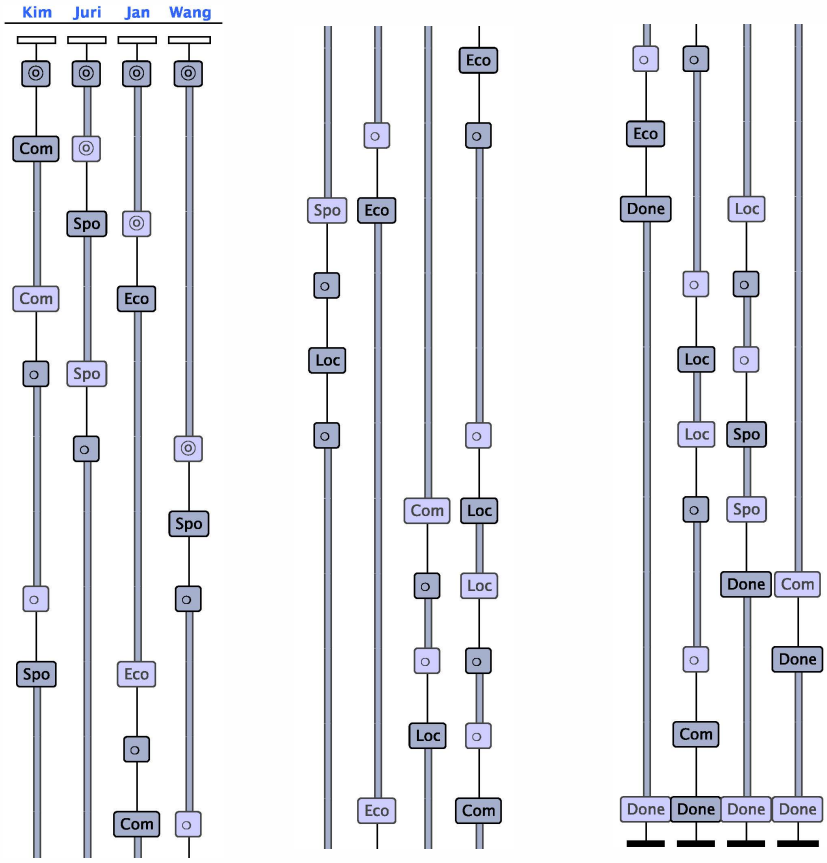
(i) Kim is impatient and does not want to wait more than 5 minutes between finishing a section and start reading the next one. Modify the original model `Newspaper.xml` accordingly. Use UPPAAL to analyze whether there still exists a schedule and if so what the minimum completion time is.

Please upload to Digital Exam your modified UPPAAL model in a file with name `Newspaper5.xml`.

Solution sketch

(a) ϕ_1: `E<> Kim.Done and Juri.Done and Wang.Done and Jan.Done and time<=50`.

(b) The query is satisfied. The Message Sequence Chart of an optimal schedule (which also satisfies ϕ_1) is shown below.



(c) ϕ_2: `E<> Kim.Done and Juri.Done and Wang.Done and Jan.Done and time<=20`.

(d) The query is not satisfied.

(e) The minimum completion time is 37 minutes. One can obtain this by doing a binary search on the upper bound of the time using ϕ_1 as a basis, or one can ask UPPAAL to generate the fastest diagnostic trace for ϕ_1 and inspecting the Symbolic Simulator for the value of `time`. The MSC of an optimal schedule is shown in b).

(f) The new minimum completion time is 38 minutes.

(g) The new minimum completion time is 37 minutes.

(h) We can model this by introducing a new global Boolean variable that blocks Jan, i.e., the first transition in Jan has a guard `eco==0 && !b`. Kim unblocks Jan by setting `b` to false when he finishes sport. Here `b` should be initialized to true (by default Booleans are initialized to false). The new minimum completion time is 52 minutes.

(i) We can model this by adding the invariant `x<=5` to the 1st, 3rd, 5th and 7th location of Kim and resetting clock `x` every time Kim finishes reading a section. The minimum completion time is 40 minutes.

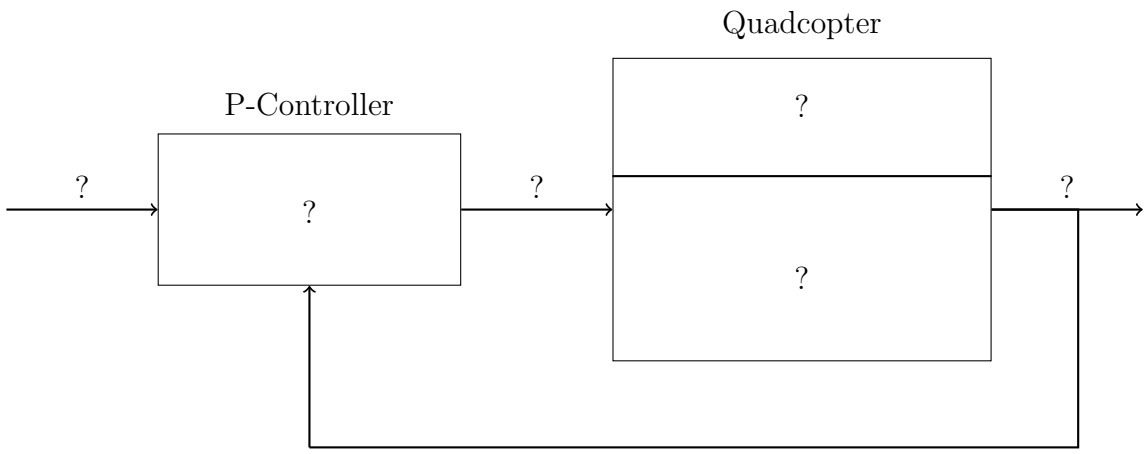
Exercise 6: Quadcopter 20 Points

Consider the quadcopter model given by

$$\frac{d}{dt}z(t) = v(t) \qquad \frac{d}{dt}v(t) = -9.81 + \frac{1}{0.5}F(t),$$

where z is the height, v the vertical velocity and F the thrust applied (for simplicity, we assume that the mass of the quadcopter is one half). Starting at position $z(0) = 0$ and $v(0) = 0$, we wish to bring the quadcopter to the height 1 within 10 time units. To this end, we next devise a controller.

(a) Complete the diagram given below so that it describes a proportional controller of the quadcopter in block diagram form.



For the following three questions, you will need to complete the provided Python script `NumericalSolverExam23.py`. You can find the Python script in Digital Exam. After finishing this exercise, please upload your modified Python script of the quadcopter in a file with name `NumericalSolverExam23completed.py`, also in case you could only solve questions b) or c).

(b) Implement the `f` method in the provided Python script so that the overall program simulates a P-controlled quadcopter. Afterwards, tune the proportional gain K_p so that the quadcopter reaches the desired height within $t = 10$ time units.

Describe the quality of your controller in terms of steady state error, settling time, overshoot and rise time.

(c) Extend the P-controller from the foregoing exercise to a Proportional-Derivative (PD) controller. To this end, derive a formula for the derivative error and extend afterwards the method `f` in the Python script.

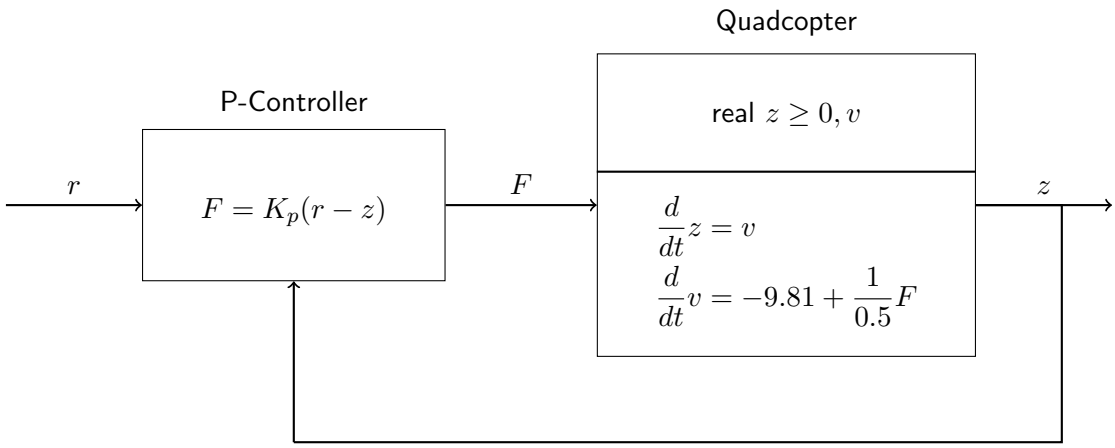
After tuning the proportional and derivative gains of the PD-controller, describe its quality in terms of steady state error, settling time, overshoot and rise time.

See next page for final questions!

(d) Extend the PD-controller from the foregoing exercise to a PID-controller. To this end, derive a formula for the integral error and extend afterwards the your python method `f`. After tuning the proportional, derivative and integral gains of the PID-controller, describe its quality in terms of steady state error, settling time, overshoot and rise time.

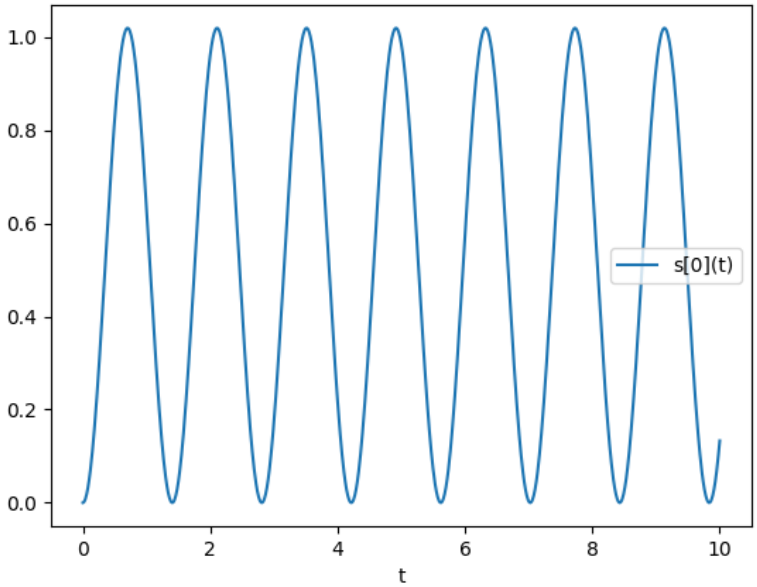
.....Solution sketch .....

(a) The block diagram is completed as follows.

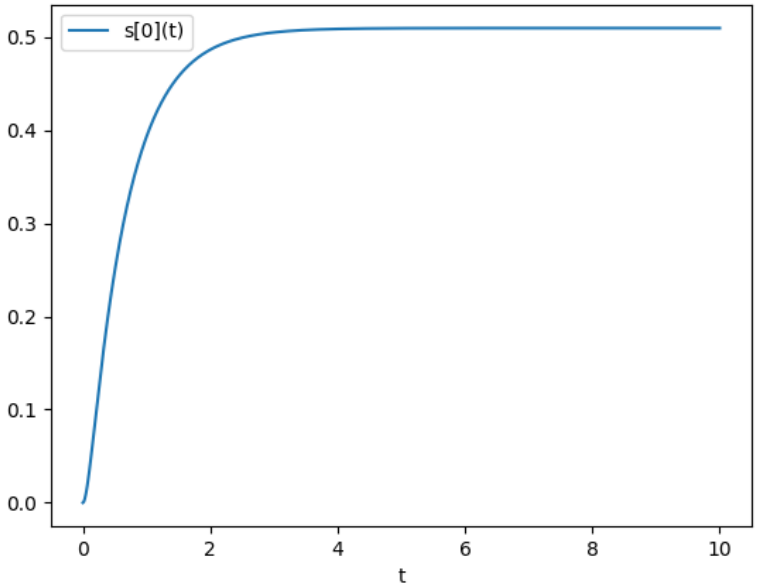


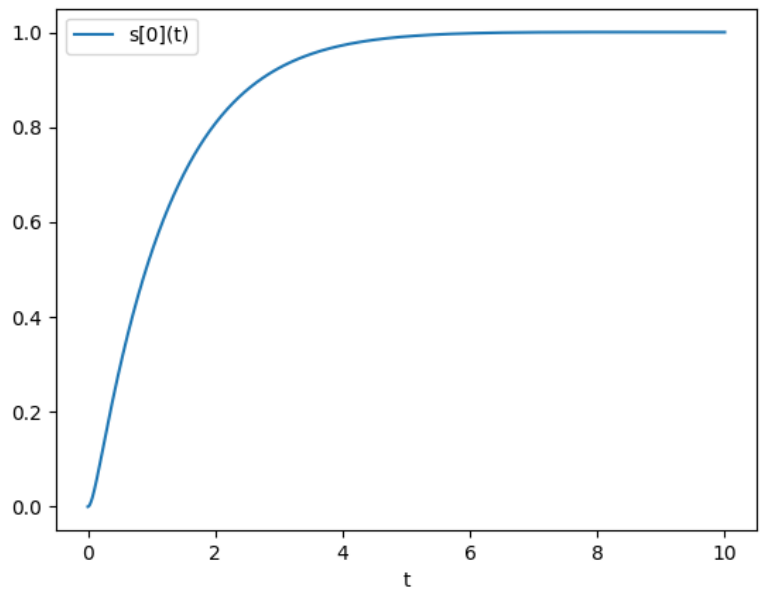
(b) For formulas and code, please see d). As for the interpretation of the results: We note that picking a sufficiently large $K_p \geq 9.81$ (e.g., 10) we can attain the reference height of 1, see figure below. The rise time is roughly 0.8 time units, which is within the limit of 10 time units (higher values of K_p will result in faster rise times). Unfortunately, there is an overshooting error for any $K_p > 9.81$. Furthermore, there is an oscillation between zero and the reference value 1. The system is stable, but not asymptotically, hence there is no steady state. Therefore, there is no steady state error and no (or infinite) settling time.

Oscillation errors can be addressed by extending the P-controller to a PD-controller.



(c) For formulas and code, please see d). Note that the derivative error is equivalent to $\frac{d}{dt}e(t) = \frac{d}{dt}(r - z(t)) = -v(t)$. The figure below is obtained with $K_p = 10$ and $K_d = 7$. By considering an extension to a PD-controller, we can eliminate the oscillation and gain an asymptotically stable controlled system with, in our case, a settling time of 4 time units, which is again below the limit of 10 time units. Unfortunately, we are left with a steady-state error (the higher K_p, the smaller this error). This can be addressed by adding an integral part to the control, yielding a PID-controller. Depending on the values of $K_p = 10$ and $K_d = 7$, one might obtain a response without overshoot and rising time (see the figure), or one with some overshoot and a quick rising time.



(d) Instead of being given by $I(t) = \int e(t)\mathrm{d}t$, the integral error enjoys the differential equation $\frac{d}{dt}I(t) = e(t) = r - z(t)$. This results an in additional state variable (so we need to adapt the initial state declaration in `main` as well!). Using a PID-controller whose code is given below, we can achieve satisfying performance by eliminating overshooting and steady-state errors, and having a rise time = settling time of roughly 7 time units, all within the required time frame. (Note that the PD-controller is obtained by letting $K_i = 0$, while the P-controller if $K_i = K_d = 0$.)

```
def f(x, t):
    # Implements the simplified quadcopter model
    m = 0.5
    g = 9.81

    Kp = 10
    Kd = 7
    Ki = 4
    r = 1

    F = Kp * (r - x[0]) + Ki * x[2] - Kd * x[1]

    der_x = x[1]
    der_v = F / m - g
    der_e = r - x[0]
    return [der_x, der_v, der_e]

def main():
    x0 = [0, 0]  # Initial state for answer b) and c), as provided
                                                in given script.
    x0 = [0, 0, 0]  # Initial state for answer d).
    ...
```